AI Tools for Economics Research

Session 3 — How it goes wrong
Monday 22 June 2026
10:30–12:00 · 14:30–16:00
Banco de Portugal
João B. Sousa · Nova SBE

TODAY

What goes wrong, and how to catch it.

Friday's session showed the agent producing usable work. This one looks at how it fails, and the habits that minimize those failures.

01

How it goes wrong.

THE STARTING POINT

Fluency is the default. Accuracy is not.

The model returns an answer that fits the request. When you have given it everything it needs, the fit is usually correct. When something is missing or unclear, fit wins — and fills the gap with something plausible.

A made-up answer (a "hallucination") is not a malfunction. It is the model doing its normal job.

Where the gaps come from.

A bad PDF read part of a PDF is missed (e.g. a table), so the model never sees it.

A partial read the agent reads the first rows of a large file; a later break is missed.

A lost summary an earlier file gets summarised, and the detail goes with it.

A habit from training the model fills a gap with whatever is usually true.

FRIDAY, REVISITED

Why the quote-with-page rule is not enough.

The rule made each claim checkable. But it cannot stop a made-up answer when the paper never reached the model: one wrong file name or a failed read, and the model still produces a fluent sentence with a believable page number — from memory (training), not from the PDF.

The rule makes a made-up answer easy to catch, not impossible to produce. The page still warrants opening.

LIVE DEMO

One prompt, two models.

Friday's six-paper folder, plus two papers that are not in it. Every cell must be filled. The same prompt runs first on the default model, then on a smaller one.

Live demonstration.

```
papers/inflation_expectations/
has six PDFs. I need a
complete comparison table,
one row per paper: Paper |
Main estimate | Supporting
quote | Page. Include the six
papers in the folder plus
Mankiw and Reis (2002) and
Coibion and Gorodnichenko
(2012). Every cell must be
filled — no blanks, the table
goes straight into slides.
Abstract/intro of each PDF
is enough.
```

WHAT CAME BACK

Same gap, two answers.

Default model quotes with page numbers; flags the two missing papers as unverifiable.

Smaller model a full table with no flags — quotes and pages for papers it never opened.

A tendency, not a guarantee — the cited page still warrants opening.

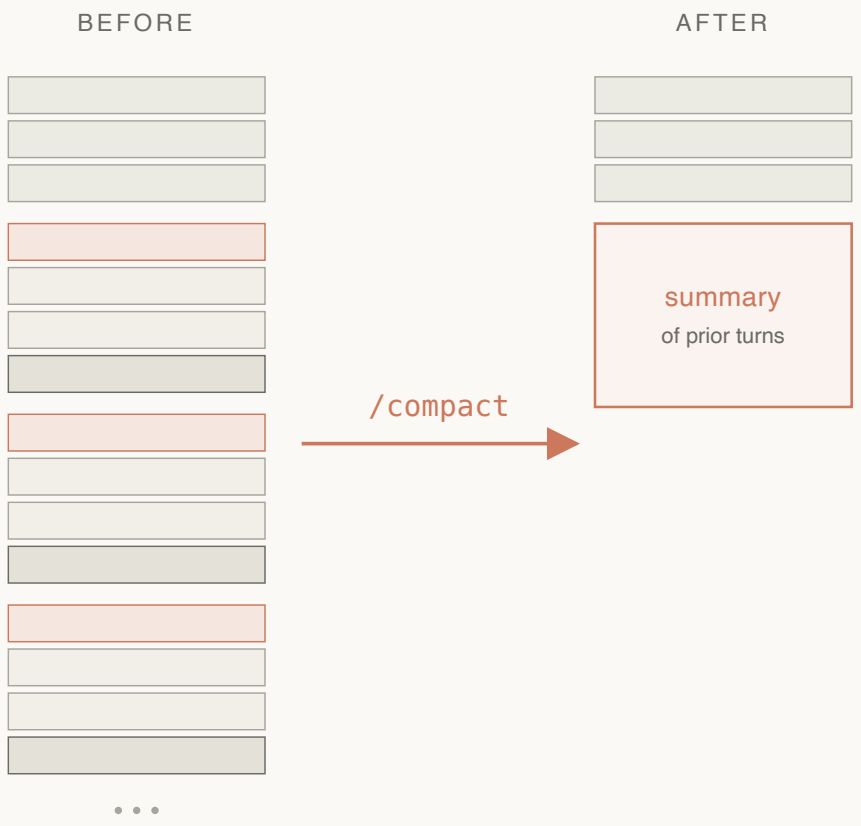
CONTEXT DRIFT

Two reasons it happens.

Nothing gets removed — old results and earlier wrong guesses stay in view and pull later answers toward them.

The latest step wins — the most recent result crowds out the original task, which has dropped out of view.

Both come from the same thing: the conversation only grows, and nothing leaves it.



preamble stays · conversation collapses into a summary

WHAT MAKES IT WORSE

Two things that make it worse.

Automatic summarising when the conversation gets long, the tool quietly replaces the earlier part with a summary. The summary loses detail, and you did not write it.

Defending its first answer once the model has settled on a reading, later answers tend to defend it rather than recheck it.

The signs: answers stop matching the question, or the same correction is needed twice.

THREE MORE, IN RESEARCH WORK

Failures that survive a glance.

Quiet code changes a fix that also reformats, renames, or "improves" a line you never asked about.

Invented data a missing value filled in by guessing, or a series stretched past where the data ends.

Confident wrong answers the tone is the same whether the model knows or is guessing.

02

Verification habits.

THE IDEA

Each check matches its failure.

Each kind of failure has a quick check that catches it. The habit is to run that check before the result reaches your draft, rather than trusting how good the answer looks.

HABIT 1

Every change is worth reading.

The agent shows each edit before it takes effect, and it repays reading. A one-line change is a one-line read; a hundred-line "fix" to a one-line problem is the signal to stop.

Version control is the safety net — it records each change and can undo it. But reading the change before you approve it is the cheapest place to catch a quiet edit.

HABIT 2

Every data edit lives in code.

The agent writes the code, not the numbers each change to the data is a line you can read, not a value typed straight into a cell.

Reading the code covers the whole dataset one read of the steps stands in for every row, where checking values one by one never could.

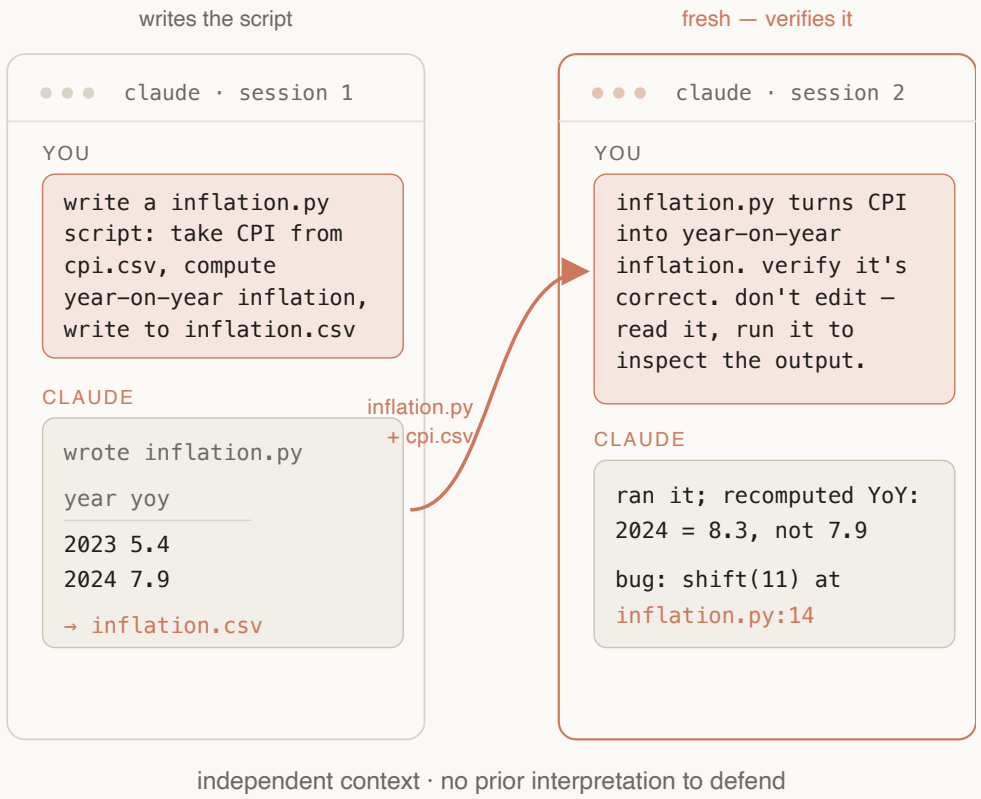
Friday's `shift(11)` error a one-line mistake, caught by reading the code rather than scanning the numbers it produced.

HABIT 3

A fresh agent makes the better reviewer.

The session that wrote the code is the worst to review it — it will defend its own reading. A new session, told what the code should do, can read and run it to check, not change it.

A clean start, nothing to defend — like a second referee.



HABIT 4

Every claim can point to its evidence.

The agent can be asked to quote the passage it relied on and say where it came from — for a paper, the page; for a number, the file and line. Then you can open it. The quote does not prove the claim; it shows exactly what to check.

Back to §1: the rule turns a summary you cannot check into one you can.

END OF MORNING

Where we are.

1. A made-up answer is the default when the input is incomplete — the model aims for fluency, not accuracy.
2. The conversation drifts as it grows — summarising and self-defence make it worse.
3. Each check matches its failure — the change read, the data edits in code, a fresh agent consulted, every claim tied to its source.

After lunch: writing prompts, then what it costs.

BREAK

14:30

Back after lunch.

03 · AFTERNOON

Writing better prompts.

THE LEVER, AGAIN

Most failures come from a vague request.

A vague request gets a vague result. A tightly defined task is the cheapest defence against both made-up answers and drift.

ONE THING AT A TIME

One task at a time.

A request with one goal produces a change you can review and a result you can check. A request with five goals produces a wall of changes and no clean place to stop.

A request with an "and" in it is often two requests.

SAY IT UP FRONT

The prompt patterns from Friday.

Say what the output should look like — columns, file names, types — name what you want back.

"Do not edit" — keep finding the problem and fixing it in separate steps.

"Stop and ask if ambiguous" — turns a silent wrong guess into a question.

Quote with the source — so every claim can be checked.

WHEN TO RESTART

Four signals to start fresh.

The topic changes — the earlier conversation is now just noise.

Corrected twice — the same fix needed again means it is stuck on its first answer.

Answers stop matching — the reply no longer fits the question; it has drifted.

About to check the work — a review needs a fresh start.

VAGUE

```
clean up the inflation
data and make it nice
for the paper
```

No format, no file names, no stopping rule. The gaps get filled with guesses.

SCOPED

```
From data/raw/cpi.csv compute
YoY inflation. Write long format
to data/processed/inflation.csv.
Stop and ask if the column is
ambiguous. Do not plot yet.
```

Format, file name, stopping rule, one task.

04

What it costs.

Why this is cheaper than the chat website.

The fixed part at the start — the instructions, your project notes, the files already read, the earlier exchange — is remembered after the first turn. Later turns reuse it for a fraction of the cost.

By default, what is remembered lasts 5 minutes after each use (an hour is optional). Reusing it costs roughly a tenth of sending it again.

WHAT FOLLOWS

Three things that follow.

Staying in one session — far cheaper than starting over from scratch.

Reading once, asking many — a file read once answers ten questions, not ten reads.

The 5-minute gap — pause longer and what was remembered is dropped; the next turn pays full.

WHAT AN EDIT COSTS

Change something mid-session and everything after it is billed in full again, not reused at a tenth of the price.

e.g. a tweak to CLAUDE.md after reading six papers re-bills all six.

MODEL CHOICE

When a smaller model is enough.

Use the largest model for the hard thinking — finding a subtle bug, working through a derivation, planning a multi-step task. Routine work — renaming, reformatting, pulling out fields, simple edits — runs fine on a smaller, cheaper, faster one.

You can switch models in the middle of a session. The model matches how hard the step is, not the whole project.

LIVE DEMO

What the cost meter shows.

One session, two ways to lower the bill. Read `cpi.csv` once, then ask several questions — `/cost` shows the first turn paying to remember the file, later turns reusing it for a fraction. Then switch to a smaller model for a routine edit: same change, lower cost.

```
read data/raw/cpi.csv
→ /cost · turn 1 writes cache
mean CPI by decade
flag the outlier years
→ /cost · later turns ≈ 1/10
──── mechanical edit ────
/model haiku
rename cpi → cpi_index in scripts
→ /cost · same diff, cheaper
```

Live demonstration. The morning's demo was the limit — there the small model failed; a rename is where it fits.

END OF SESSION

Four habits for tomorrow.

1. Fluency assumed, accuracy verified — the output looks finished whether or not it is.
2. A check for each failure — the change, the code, a fresh agent, a cited source.
3. Tight focus, frequent fresh starts — one task at a time; start over when it drifts or needs review.
4. Staying in one session where possible — reusing what is read makes the long session the cheap one.

See you tomorrow.

joao.sousa@novasbe.pt